FIAP × CLARO

Challenge Sprint 4

Claro Omnichannel — melhorias implementadas na solução

Vídeo de demonstração: [COLE AQUI O LINK PÚBLICO DO VÍDEO]

Repositório: https://github.com/mathcerq/claro-omnichannel

Equipe:

- [Nome completo] — RM [00000]

Outubro de 2026

1. A solução

O desafio da Claro é a fragmentação do atendimento: quando o cliente abre um chamado em um canal e é transferido para outro, o histórico se perde e ele precisa repetir tudo. O Claro Omnichannel resolve isso com um único protocolo (CLR-AAAA-NNNNN) e uma única linha do tempo, compartilhada por todos os canais e visível em tempo real para o cliente e para o atendente.

A solução tem três partes: o app do cliente (React PWA, com WebChat), o painel do atendente (React) e o backend (NestJS, Prisma, WebSocket), que integra os canais WebChat e Telegram (reais) e telefone (simulado de propósito), além de uma assistente virtual com IA local. A arquitetura completa está no Anexo A.

2. Como as melhorias foram definidas

As melhorias partiram de três referências: a orientação do professor de que o ideal é uma solução totalmente funcional, aceitando simulação apenas quando necessário; os critérios de avaliação da Challenge (protótipo 100% funcional, requisitos funcionais e não funcionais, tecnologias adequadas, maturidade técnica, alinhamento com a Claro, criatividade e inovação); e o objetivo do desafio da Claro de manter o histórico ao trocar de canal.

Com essas referências, a equipe revisou a versão anterior do sistema funcionalidade por funcionalidade, inclusive com testes automatizados e verificações diretas na API, e priorizou os cinco pontos abaixo. As imagens de "antes" foram capturadas executando a versão anterior do código (commit 9996156 do repositório) e as de "depois", a versão atual.

Resumo das cinco melhorias

#	Melhoria	Antes	Depois
1	**Canal real (Telegram)**	WhatsApp e telefone simulados por uma rota técnica	Bot real do Telegram com vínculo seguro e a mesma timeline; telefone mock explícito
2	**Tempo real**	Mensagens só apareciam recarregando a página; sessão caía a cada 15 min	Mensagens ao vivo, entrega confirmada, recuperação ao reconectar, sessão renovada
3	**Assistente virtual (IA)**	Todo chamado ia para a fila humana	IA local resolve o nível 1 e transfere com resumo; métricas de resolução
4	**Segurança e LGPD**	Cadastro como ADMIN, atendente via tudo, 2FA reutilizável	Menor privilégio, anti-replay, auditoria, exportação de dados
5	**Interface do usuário**	2FA em tela branca, F5 voltava ao início, canais fictícios	QR do 2FA, sessão mantida, telas de Telegram/IA, indicadores real/mock

Melhoria 1 — Canal real: integração com o Telegram

Como estava (antes)

- WhatsApp e telefone eram apenas simulados: uma rota técnica (/simulate) inseria na conversa um texto com o prefixo "[WHATSAPP Simulado]". Nenhum canal externo funcionava de verdade.
- O app do cliente oferecia canais que não existiam (WhatsApp, Telefone, E-mail) ao abrir um chamado pelo próprio WebChat.

O que foi feito (depois)

- Bot real do Telegram (@claro_omnichannel_mathcerq_bot) integrado pela Telegram Bot API, com long polling (funciona sem expor o servidor à internet).
- Vínculo seguro: o cliente, logado no app, gera um código de uso único (válido por 10 minutos, com deep link e QR Code). Isso impede que alguém se passe por outro cliente.
- Pelo Telegram, o cliente abre chamados (/novo ou texto livre), acompanha protocolos, vê o histórico de todos os canais (/historico) e recebe as respostas do atendente, com confirmação de entrega no painel.
- O telefone continua simulado, mas de forma explícita (rótulo "mock" no painel e no código). Os canais são plugáveis: um novo canal, como o WhatsApp Business, entra sem mudar o núcleo do sistema.
- O formulário de novo chamado não oferece mais canais fictícios.

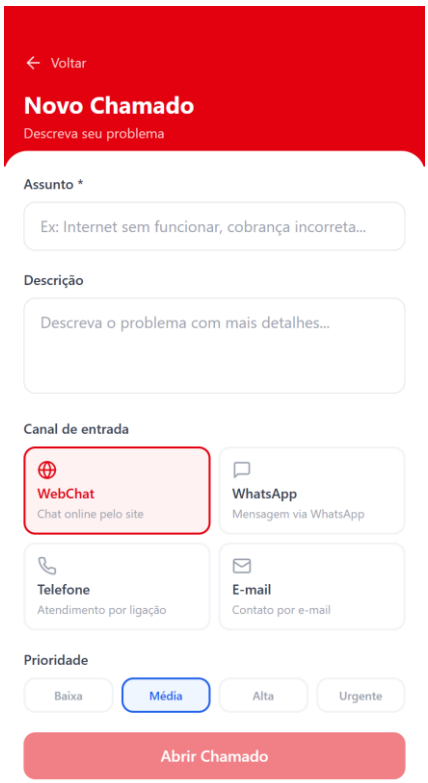
Justificativa

Atende diretamente à orientação do professor ("o ideal é tudo funcional, mock só quando necessário") e ao critério de protótipo totalmente funcional. O problema da Claro é justamente a troca de canal; demonstrá-la com um canal externo real torna a solução verificável.

O Telegram foi escolhido porque a API do WhatsApp Business exige conta comercial verificada pela Meta, o que não é viável para o projeto. A arquitetura de canais plugáveis deixa o WhatsApp pronto para ser adicionado.

Evidências

ANTES



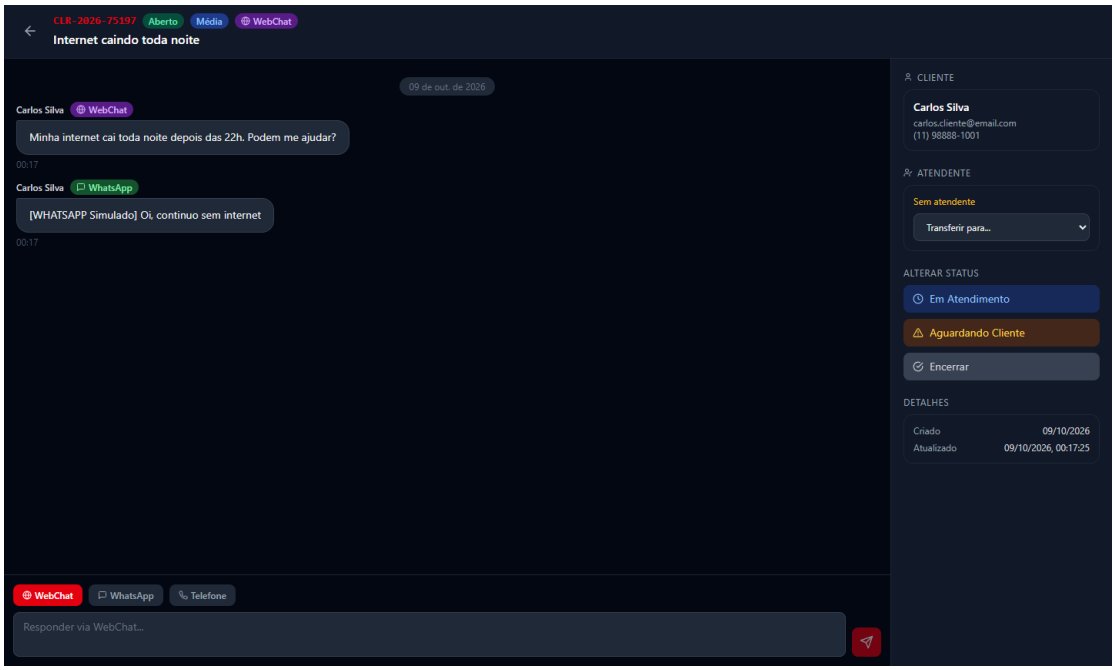
Novo chamado com canais que não existiam

DEPOIS



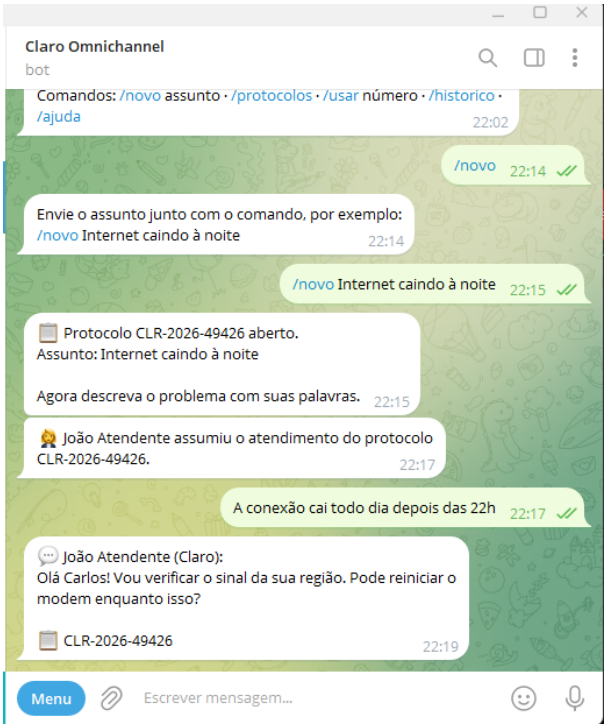
Novo chamado sem canais fictícios

ANTES

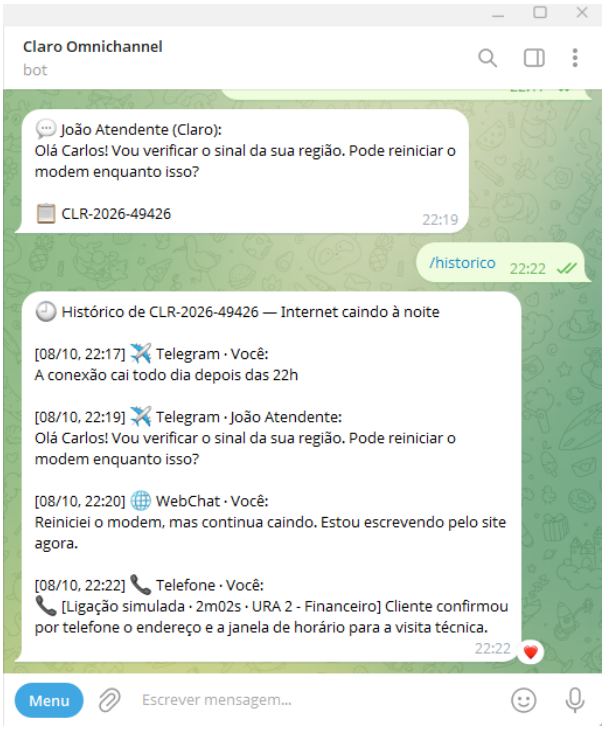


Painel: o "WhatsApp" era um texto inserido pela API com o prefixo "[WHATSAPP Simulado]"

DEPOIS



Telegram real: chamado aberto pelo bot e resposta do atendente entregue



/historico: Telegram, WebChat e telefone no mesmo protocolo

Melhoria 2 — Atualização em tempo real

Como estava (antes)

- O servidor WebSocket existia, mas nenhum evento era disparado: uma mensagem nova do cliente só aparecia para o atendente depois de recarregar a página (e vice-versa).
- A sessão expirava a cada 15 minutos sem renovação: o atendente era desconectado no meio do atendimento.

O que foi feito (depois)

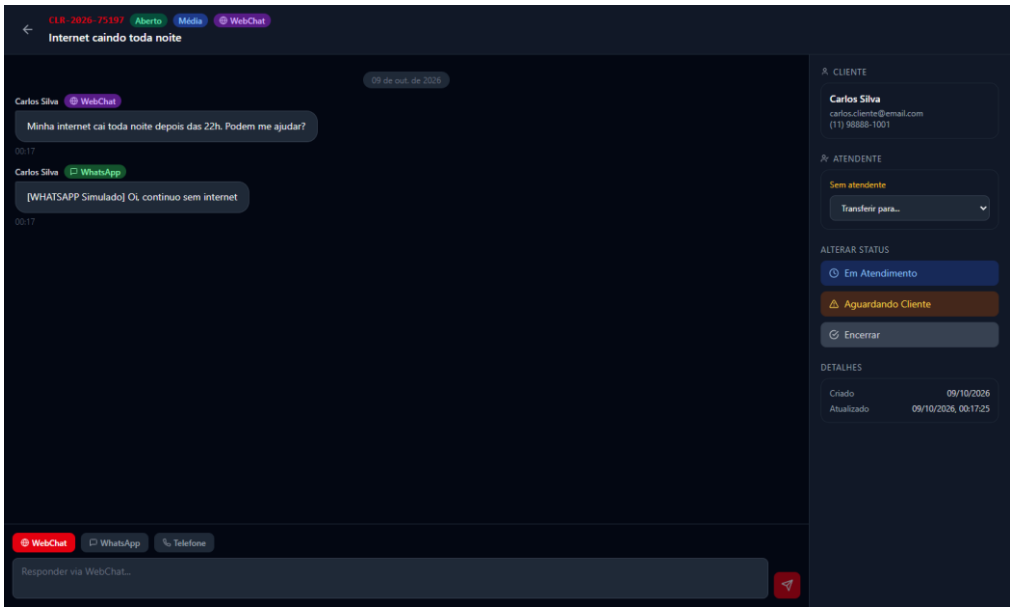
- O núcleo publica eventos (mensagem criada, protocolo alterado) e o WebSocket os entrega às salas certas: o atendente vê a mensagem do cliente na hora, em qualquer canal.
- Status de entrega por mensagem ("entregue no Telegram") e indicador "digitando…" da assistente.
- Recuperação automática: se a conexão cai (servidor reiniciando, Wi-Fi oscilando), a tela busca o que mudou ao reconectar.
- Renovação automática do login com refresh token: a sessão não cai mais a cada 15 minutos.

Justificativa

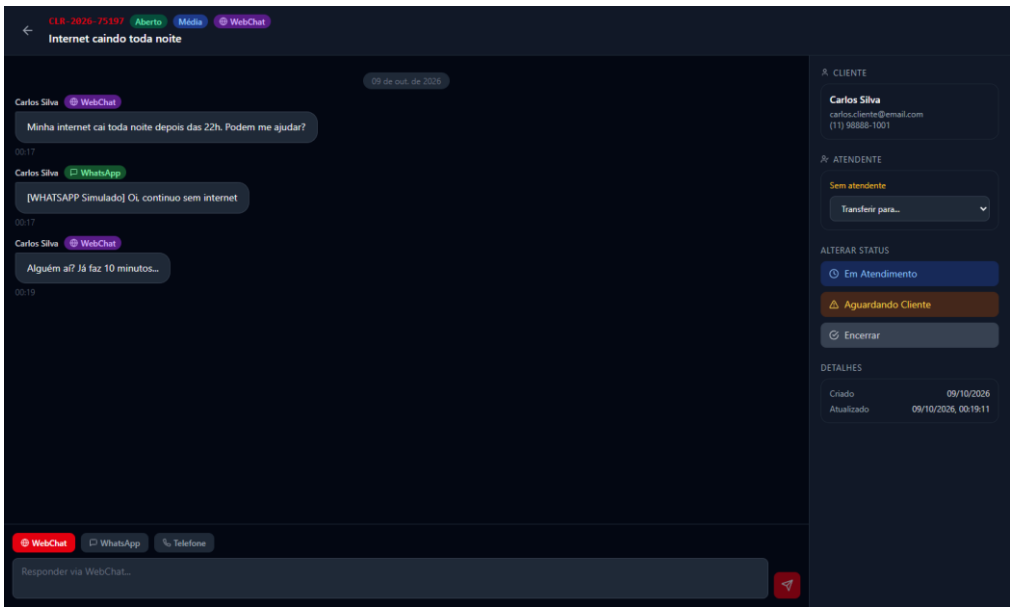
Atendimento omnichannel depende de o atendente enxergar o cliente no momento em que ele escreve; sem isso, o tempo médio de atendimento aumenta e o cliente fica sem resposta. É também um requisito não funcional de usabilidade e confiabilidade, um dos critérios de avaliação.

Evidências

ANTES

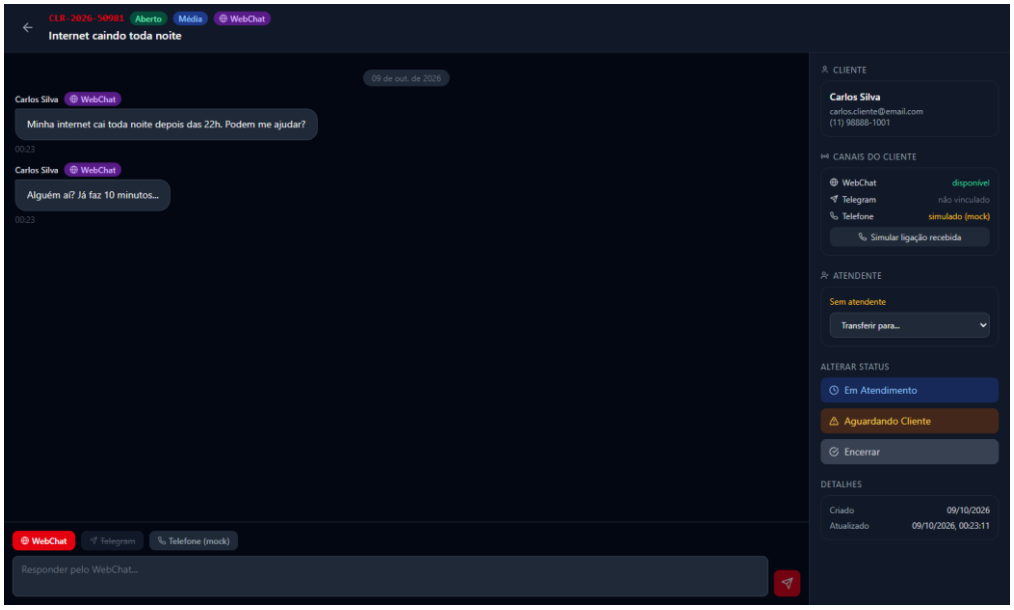


O cliente enviou "Alguém aí? Já faz 10 minutos…", mas o painel não mostra nada…

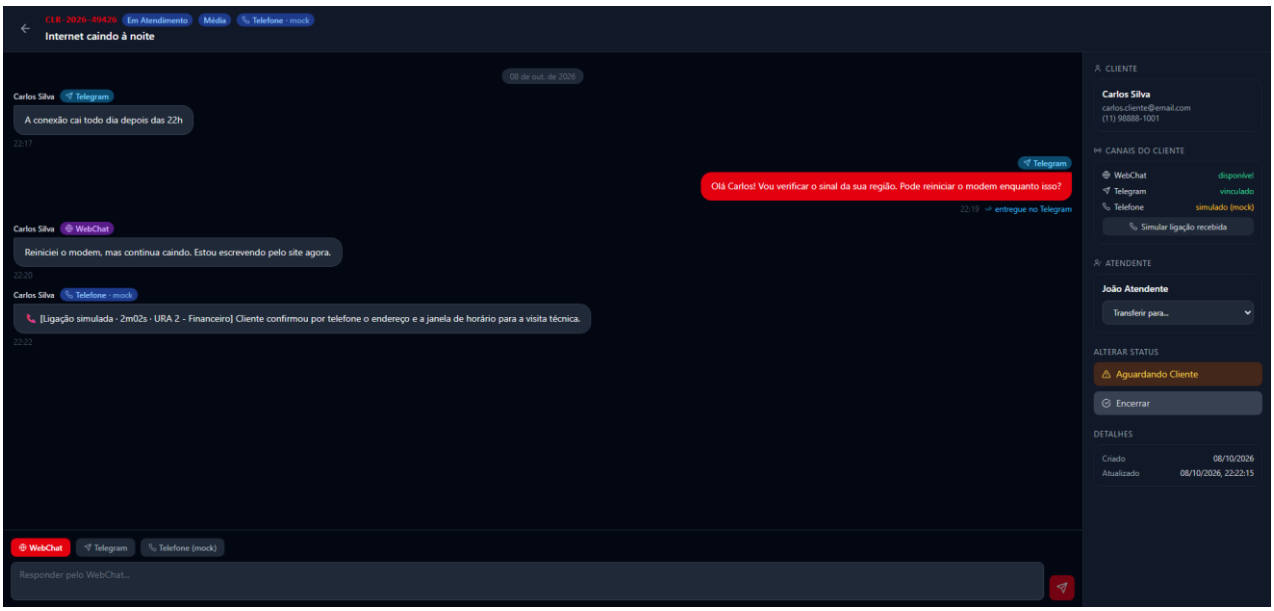


...a mensagem só aparece depois de recarregar a página.

DEPOIS



A mesma mensagem aparece sozinha no painel, sem recarregar



Timeline com badge por canal e "✓entregue no Telegram" na resposta do atendente

Melhoria 3 — Assistente virtual com IA (atendimento nível 1)

Como estava (antes)

- Todo chamado ia direto para a fila dos atendentes humanos; o cliente escrevia e ficava esperando.
- Não havia nenhuma automação nem indicador de quanto do atendimento poderia ser resolvido sem um humano.

O que foi feito (depois)

- A Clara, assistente virtual, atende primeiro no WebChat e no Telegram, na mesma timeline do protocolo.
- Usa IA generativa rodando localmente (modelo aberto Qwen3 4B via Ollama), sem custo e sem enviar dados para terceiros.
- Resolve sozinha os casos de nível 1 chamando ferramentas: fatura e 2ª via, verificação de instabilidade na região, diagnóstico e reinício remoto do modem, agendamento de visita técnica. Os sistemas internos da Claro são simulados.
- Transfere para um humano quando necessário (pedido de atendente, cancelamento, contestação, Procon), deixando motivo e resumo do caso no painel: o atendente não precisa perguntar tudo de novo.
- Decisões críticas são regras no código, não do modelo. Se a IA ficar indisponível, um motor de regras assume automaticamente.
- Painel com selos "IA atendendo / Transferido pela IA / Resolvido pela IA" e métricas, incluindo a taxa de resolução sem humano.

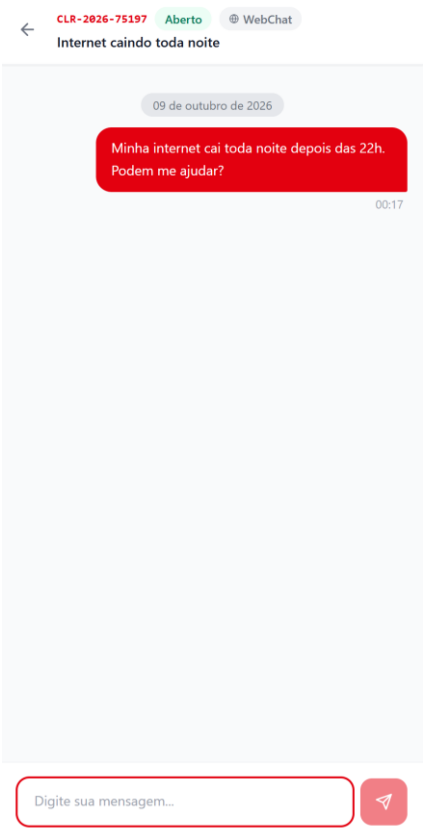
Justificativa

Alinha a solução ao objetivo da Claro de reduzir filas e tempo médio de atendimento e de liberar os atendentes para os casos complexos. Atende ao critério de criatividade e inovação com uma abordagem responsável: a IA roda localmente (LGPD), é sempre identificada como IA, e o cliente pode pedir um humano a qualquer momento.

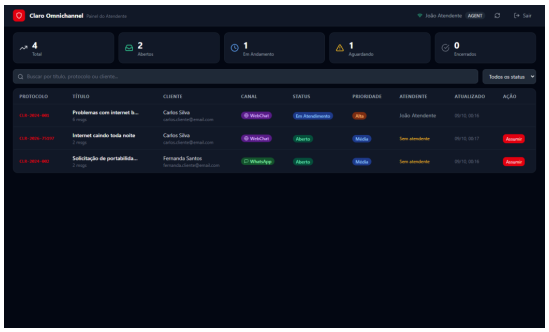
Foram comparados três modelos em cenários reais (fatura, internet, pedido de humano e contestação). O escolhido acertou 4 de 4; um modelo mais rápido acertou 0 de 4 por prometer ações sem executá-las.

Evidências

ANTES

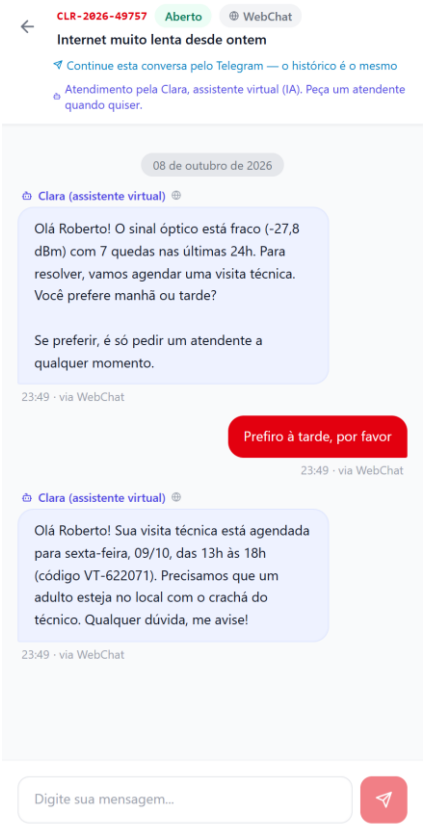


O cliente escreve e ninguém responde até um atendente pegar o chamado

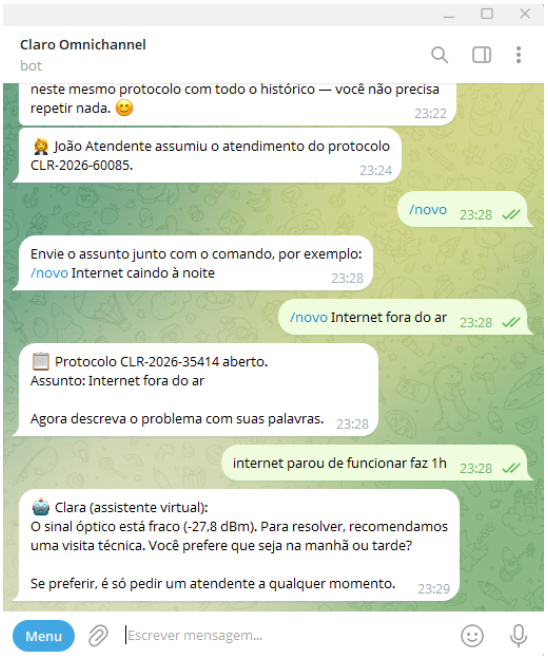


Dashboard sem automação nem métricas de IA

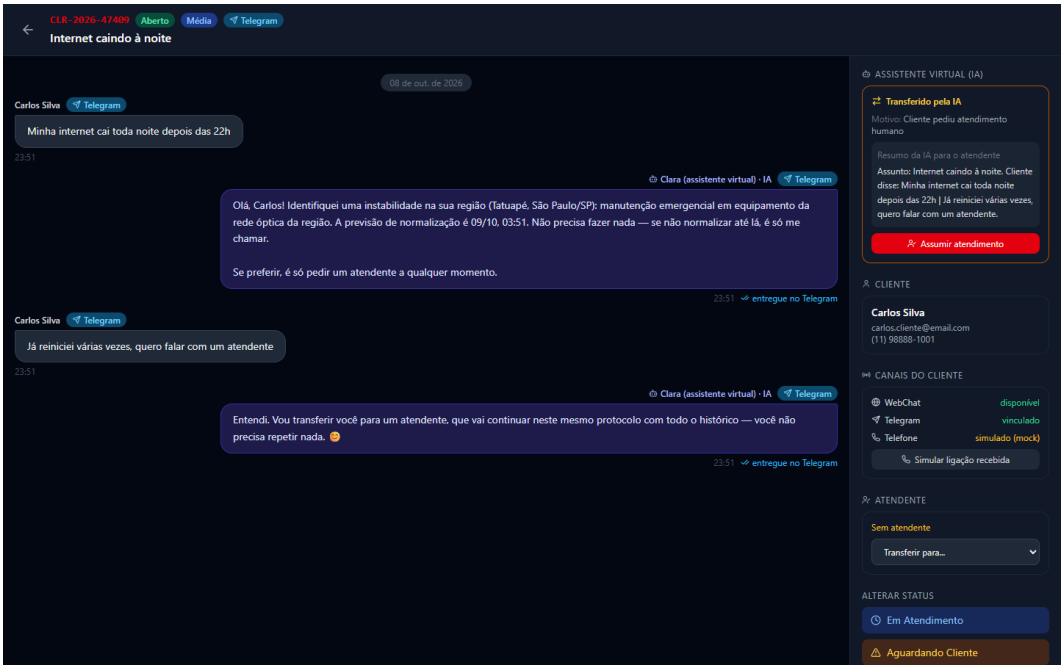
DEPOIS



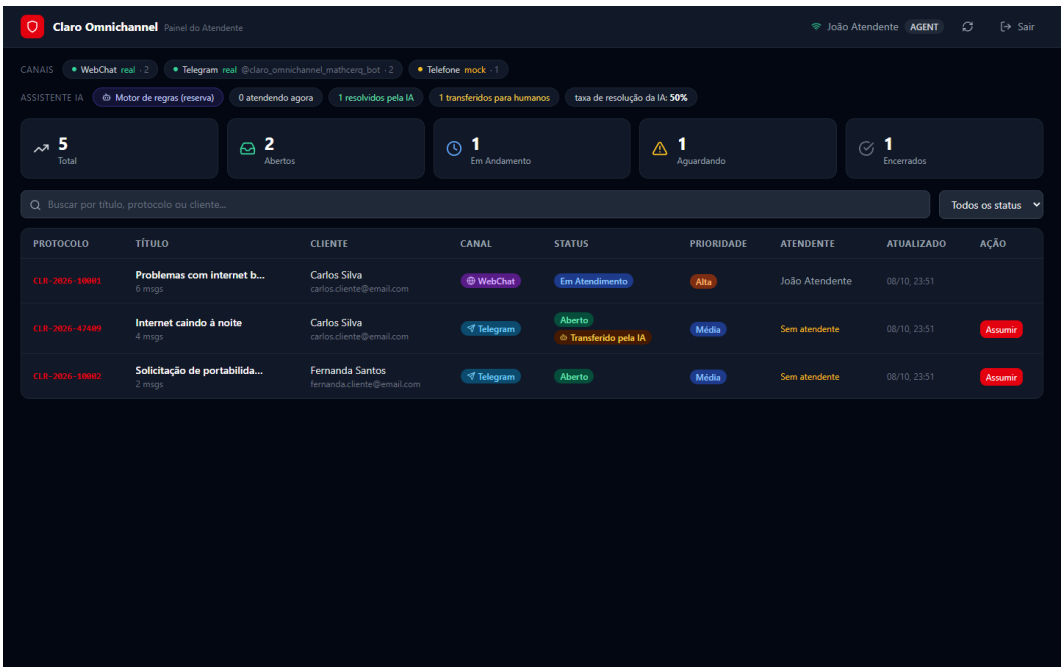
WebChat: a Clara diagnostica o sinal fraco e agenda a visita técnica



Telegram: a Clara diagnostica e transfere para o atendente



Painel: transferência feita pela IA, com motivo e resumo do caso para o atendente



Dashboard: canais reais/mock, métricas da IA e selo "Transferido pela IA"

Melhoria 4 — Segurança e LGPD

Como estava (antes)

- O cadastro público aceitava o campo de perfil: qualquer pessoa criava uma conta de ADMINISTRADOR (resposta 201).
- Qualquer atendente abria qualquer protocolo pelo identificador, inclusive os de clientes de outros atendentes (resposta 200).
- As salas do WebSocket não verificavam permissão e eventos eram enviados a todos os conectados.
- O mesmo código do 2FA podia ser usado mais de uma vez, e segredos tinham valores padrão escritos no código.

O que foi feito (depois)

- Cadastro público cria sempre CLIENT (o campo de perfil é rejeitado: 400).
- Menor privilégio: o atendente acessa só a fila e os protocolos atribuídos a ele (403 nos demais); a mesma regra vale no WebSocket.
- 2FA com anti-replay (cada código vale uma vez: 401 no reuso) e API que não sobe sem os segredos configurados.
- LGPD: trilha de auditoria (logins, transferências, alterações), exportação dos dados do titular pelo app, vínculo do Telegram revogável e, do Telegram, apenas o identificador da conversa é guardado.

Justificativa

Uma operadora lida com dados pessoais de milhões de clientes. As correções atendem à LGPD (art. 6º, III, necessidade; art. 46, segurança; art. 18, direitos do titular) e aos critérios de requisitos não funcionais e maturidade técnica. As falhas foram confirmadas com chamadas reais à API da versão anterior.

Evidências

ANTES



Versão anterior: cadastro como ADMIN aceito e atendente abrindo protocolo de outro atendente

DEPOIS

```
# Versão atual — as mesmas chamadas à API

$ POST /auth/register {"name":"Invasor", ..., "role":"ADMIN"}
HTTP 400 → ["property role should not exist"]
✓ Cadastro público cria sempre CLIENT; o campo role é rejeitado.

$ GET /protocols/CLR-2026-10003 (logado como João — protocolo atribuído à Maria)
HTTP 403 → Acesso negado a este protocolo
✓ Menor privilégio (LGPD): atendente só vê a fila e os protocolos atribuídos a ele.

$ POST /auth/verify-otp (reutilizando um código 2FA já usado)
HTTP 401 → Código 2FA inválido
✓ Anti-replay: cada código TOTP vale uma única vez.
```

Versão atual: as mesmas chamadas são recusadas (400, 403) e o 2FA não aceita código reutilizado (401)

Melhoria 5 — Interface do usuário

Como estava (antes)

- Ao configurar o 2FA obrigatório, a tela do atendente ficava em branco: o QR Code nunca aparecia e o primeiro acesso pela interface era impossível.
- Atualizar a página (F5) ou abrir um link direto para um protocolo levava de volta à tela inicial.
- Não havia indicação de quais canais eram reais e quais eram simulados, nem tela para conectar canais externos.

O que foi feito (depois)

- QR Code do 2FA funcionando no painel e no app (a tela codificava a imagem inteira em vez do link do autenticador).
- A sessão é restaurada no primeiro carregamento: F5 e links diretos mantêm a página.
- Tela "Conectar Telegram" no app (código, botão "Abrir no Telegram", QR Code e desvínculo).
- Painel com indicadores de canal real/mock, card "Canais do cliente", card da assistente e "digitando…".
- Mensagens identificadas por canal ("via Telegram", "via WebChat", "via Telefone") nos dois apps.

Justificativa

A Sprint 4 pede explicitamente melhorias de interface. Os dois primeiros itens eram defeitos que impediam o uso, um deles bloqueando o primeiro acesso do atendente. Os demais deixam visível para o cliente, para o atendente e para a banca o que é real, o que é simulado e o que a IA está fazendo.

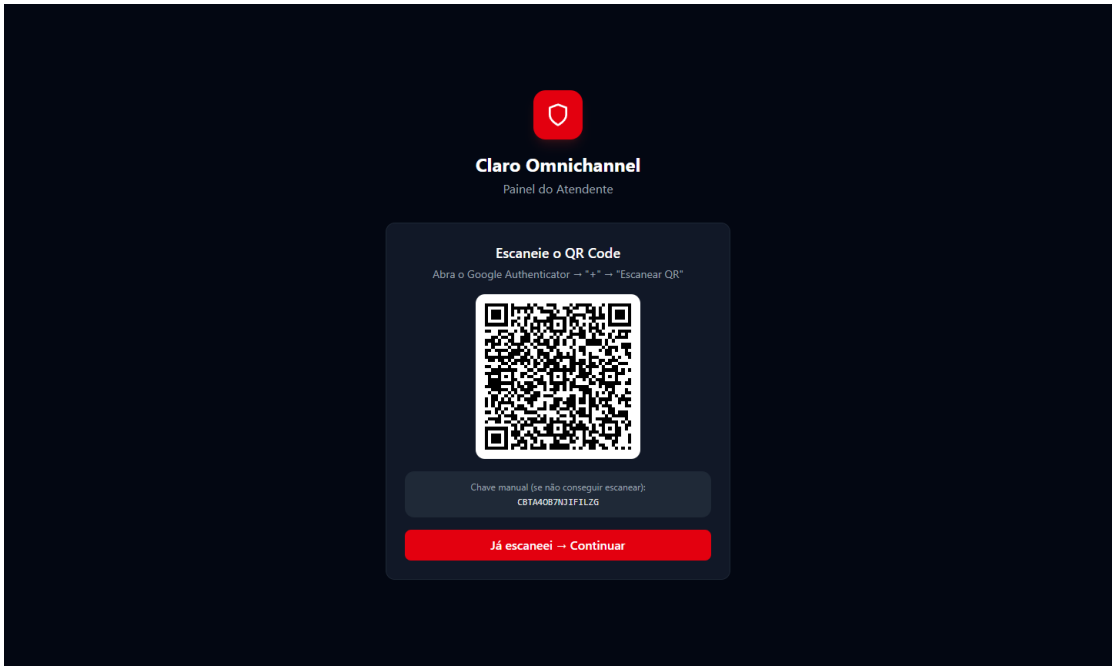
Evidências

ANTES



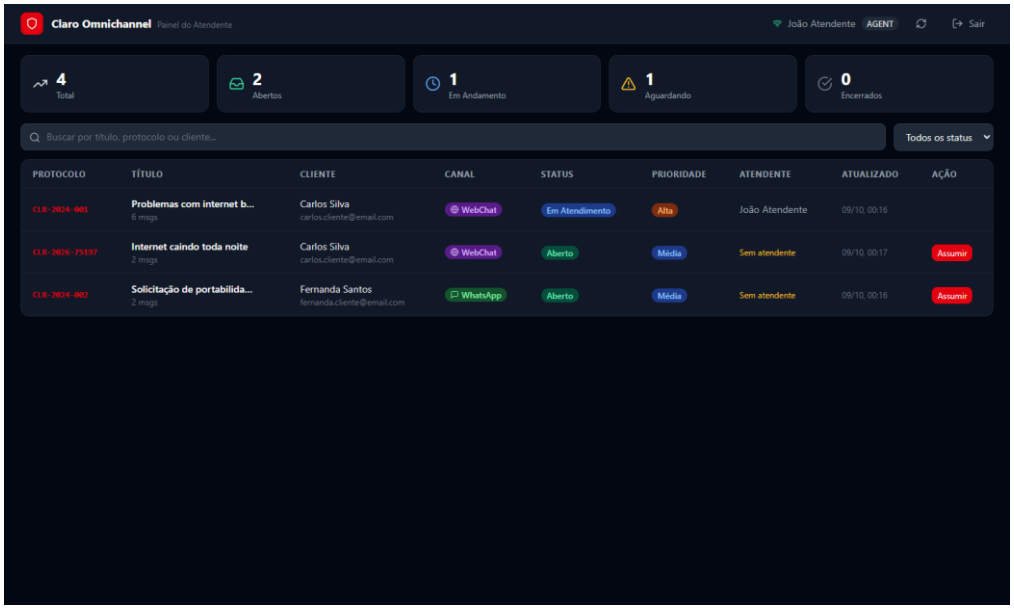
Configurar o 2FA: a tela ficava completamente em branco

DEPOIS



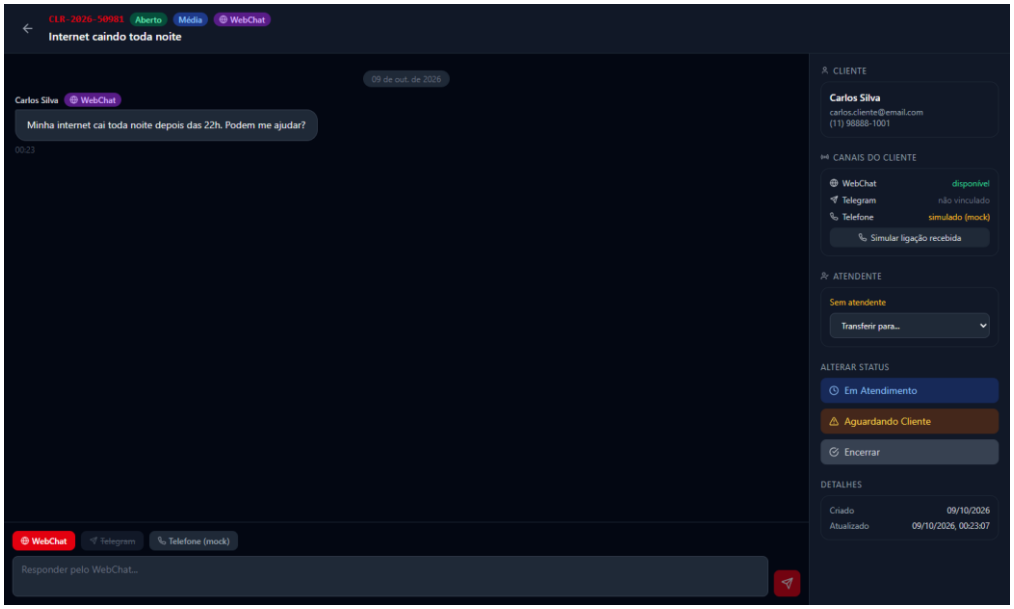
Configurar o 2FA: QR Code e chave manual exibidos

ANTES



Link direto para um protocolo: o sistema voltava para a tela inicial

DEPOIS



Link direto / F5: a página do protocolo é mantida



Nova tela do app: Telegram conectado, com opção de desvincular (LGPD)

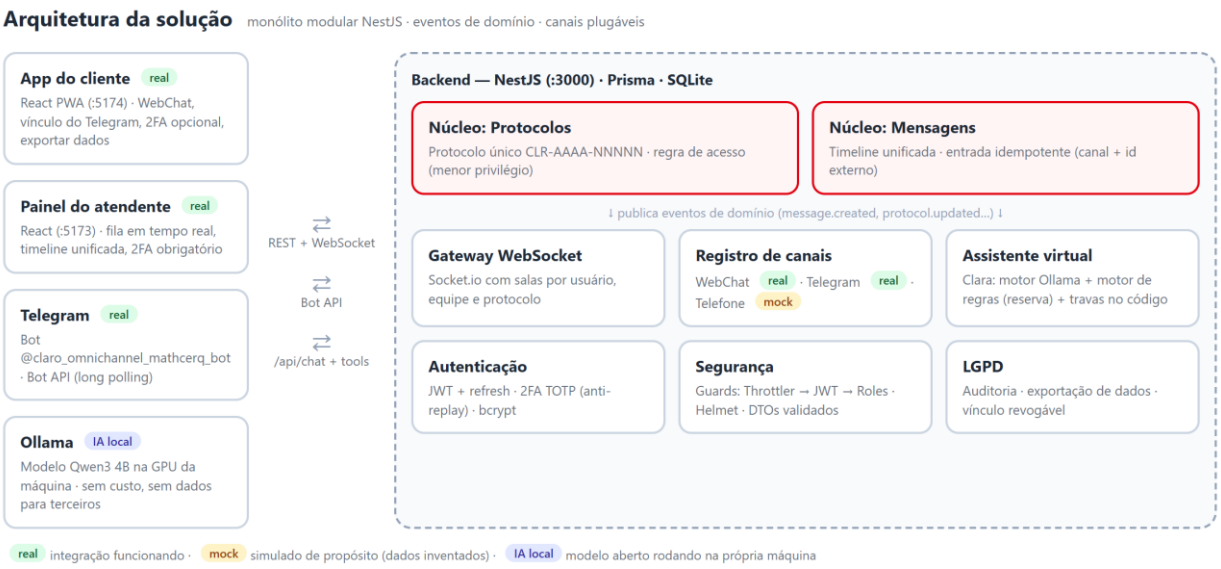
3. Como as melhorias foram validadas

Todas as melhorias foram testadas, e os testes automatizados ficam no repositório (npm test):

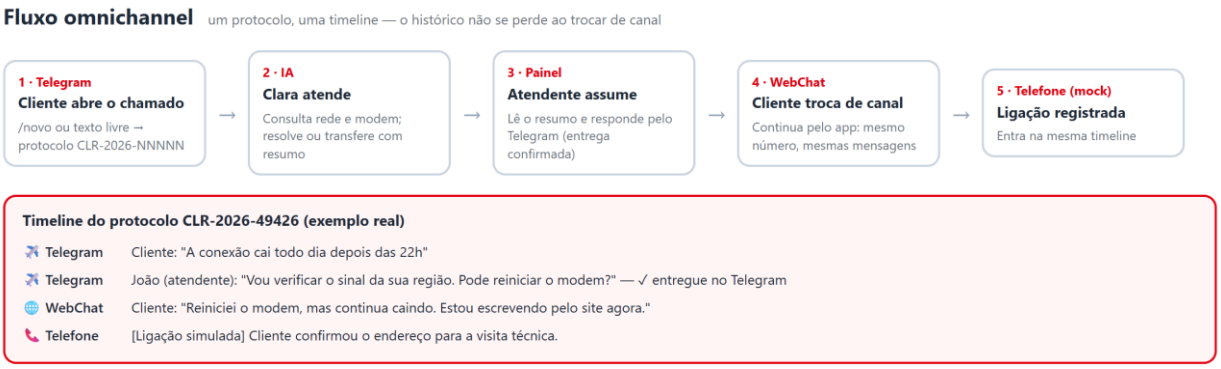
- **Testes unitários (Jest):** regras do bot, canais, regra de acesso, motores e travas da assistente.
- **E2E do Telegram:** 29 verificações com o backend real e uma Bot API simulada: vínculo, protocolo, troca de canal, entrega, idempotência, segurança e LGPD.
- **E2E da assistente:** 12 verificações: resposta no Telegram e no WebChat, transferência com resumo, saída da IA quando o humano assume, métricas.
- **E2E de interface (Playwright):** 16 verificações no navegador, incluindo a configuração do 2FA pela tela e a recuperação após o servidor reiniciar.
- **Avaliação da IA real:** cenários de conversa executados contra o modelo local (4 de 4 com o desfecho esperado).
- **Teste manual com o bot real:** jornada Telegram → atendente → WebChat → telefone executada ponta a ponta (telas reais neste documento).

Algumas correções foram feitas com um teste que falhava antes e passava depois (por exemplo, a tela em branco do 2FA e a recuperação após queda de conexão).

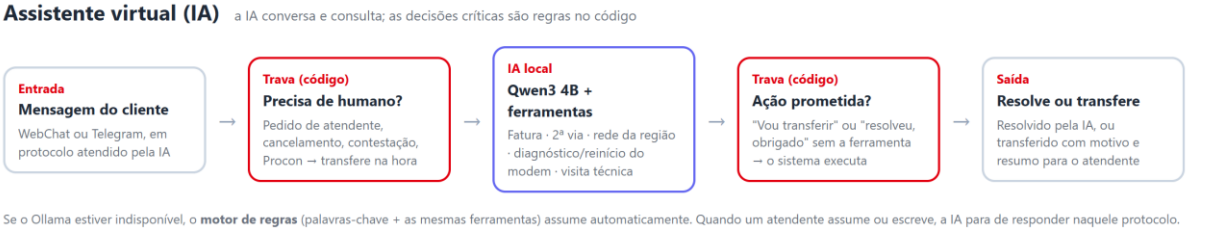
Anexo A — Arquitetura da solução



Componentes: frontends, canais, backend modular e IA local



Fluxo omnichannel: o mesmo protocolo e a mesma timeline em todos os canais



Fluxo da assistente virtual: IA com travas determinísticas

Anexo B — Tecnologias

Camada	Tecnologias
Backend	NestJS (TypeScript), Prisma ORM, SQLite, Socket.io, @nestjs/event-emitter
Frontends	React 18, Vite, Tailwind CSS, Zustand, Axios, PWA
Canais	Telegram Bot API (long polling); WebChat (REST + WebSocket); telefone simulado

IA	Ollama com o modelo aberto Qwen3 4B (local, GPU), function calling; motor de regras de reserva
Segurança	JWT + refresh token, 2FA TOTP com anti-replay, bcrypt, Helmet, rate limit, RBAC com menor privilégio
Qualidade	Jest, testes E2E com Playwright, Bot API simulada para testes